C – Practice Exercises
Year 1 / Q2
if statements
switch statements
for – while - do while loops

AUTOREN
DR. JOHANNES FARMER
DI CHRISTOPH KOHLWEG
DI URSULA RIESEL
DR. HEINZ SCHIFFERMÜLLER
DI FLORIAN SCHREIBER
DI JOHANN WURZINGER

TRANSLATION
DR. MARIA GRASSMUGG
DR. WOLFGANG PÖLZLEITNER

January 2023

Q2-1 Exercises: `if`-statements

Q2_101_LittleIfExamples
Write a C-program `LittleIfExamples` with following instructions:

a. Enter an integer. If the number is positive or zero, the square root is calculated, otherwise print a corresponding error message.

```
Please enter an integer:  4
The square root is: 2

Please enter an integer:  -4
The square root of a negative number is not possible.
```

b. Generate 3 random floating-point numbers and check, if the sum of the three numbers is bigger than their product or vice versa

```
First random number: -1.0
Second random number: 2.0
Third random number: 1.5

The sum 2.50 is bigger than the product -3.00.
```

c. Generate one random number in the range of -99 to 99. Calculate the number of digits the random number has. The following outputs are possible:

```
The random number is 9 and has one digit!
The random number is -89 and has two digits!
```

d. Check if an integer you entered is an even number and a multiple of 3. The following outputs are possible:

```
The number 12 is even and a multiple of 3
The number 4 does not meet the required conditions.
```

Addition: In the no branch, indicate which condition (odd or not a multiple of 3) is not fulfilled.

e. An entered character must be determined whether it is an upper- or lower-case letter. The following outputs are possible:

```
The character 'G' is a capital letter.
The character 'e' is a lowercase letter.
The character ',' is not a letter.
```

Addition: Check whether a letter is a vowel.

f. Enter a 4-digit year and check if the year is a leap year or not.
Note: A year is a leap year if the year is divisible by 4. If it is a multiple of 100 it is **not** a leap year. Multiples of 400, however, are leap years.
The following output is generated on the console:

```
The year 2000 is a leap year.
```

or:

```
The year 2100 is not a leap year.
```

Q2-102_NumberSort1

Write a C-program **NumberSort1**, reading 2 integers from the console and stores them in the variables number1 and number2. The two numbers are then sorted in ascending order. The variable values are swapped if the value of number1 is greater than that of number2. Print the sorted numbers in the following format:

```
Number1: 321
Number2: 123
Entered numbers: 321, 123
Sorted numbers: 123, 321
```

Q2-103_NumberSort2

Write a C-program **NumberSort2**, reading 3 integers from the console and stores them in the variables number1, number2 and number3. The three numbers are then sorted in ascending order (number1 < number2 < number3). Print the sorted numbers in the following format:

```
Number1: 8
Number2: 1
Number3: 4

Entered numbers: 8,1,4
Sorted numbers: 1,4,8
```

Q2-104_TypeOfTriangle

Write a C-program **TypeOfTriangle,** which reads in the lengths of the sides a, b and c of a triangle. The variables a, b and c are of the type integer. First check whether c is the longest side, otherwise the sides have to be swapped accordingly. Determine, what type of triangle it is:

- equilateral (a = b = c)
- isosceles (a = b ≠ c)
- right triangle (a^2 + b^2 = c²2)
- general triangle (a + b > c)
- no triangle

```
Type of Triangle:
=====================================
Length side a: 3
Length side b: 4
Length side c: 5
Type of triangle: right triangle
```

Q2-105_MinMax

Write a C-program **MinMax**, that creates 6 random numbers in the range of 10 to 50. Determine the smallest and the largest number.
Print all numbers in the following format:

```
MinMax computation
****************************************
Random numbers: 34 - 17 - 41 - 23 - 27 - 32
```

```
Largest number: 41
Smallest number: 17
```

Q2-106_SumOfNumbers

Write a C-program **SumOfNumbers**, that creates 5 random numbers in the range of -60 to 60. Sum up separately all positive and all negative numbers and store them in the variables **sumPos** and **sumNeg.** Determine the smallest and the largest number.
Print all numbers in the following format:

```
Sum of numbers
-------------------------------------------
Random numbers: 24/-16/53/4/-49
Sum of the positive numbers: 81
Sum of the negative numbers: -65
Smallest number: -49
Largest number: 53
-------------------------------------------
```

Q2-107_ Quadrant

Write a C-program **Quadrant** that inputs the x and y coordinates of a point in the plane and determines in which quadrant (I-IV) the point lies. Print all numbers in the following format:
Print following output on the console:

```
The point -3/6 is on the 2nd quadrant.
```

Q2-108_CircleLine

Write a C-program **CircleLine,** that calculates for a circle and a point whether the point is located on the circle, inside the circle, or outside.
The circle is defined by its center coordinates **xmp**, **ymp** and its radius **radius**. The point **P** is defined by its coordinates **xCoord**, **yCoord**. These five values are read in.

```
Input of the x-coordinate of the point: 4
Input of the y-coordinate of the point: 12
Input of the x-coordinate of the center point: 3
Input of the y-coordinate of the center point: 3
Enter the radius length: 5

The point (4/12) lies outside the circular line.
```

Q2-109_NumberAnalysis

Write a C-program **NumberAnalysis,** that analyses a given number. The analysis establishes the following facts:
(i) The number is positive (≥ 0) or negative (<0).
(ii) The number is even or odd.
(iii) How many digits has the number?

The number entered can be up to 5 digits.

The following output is generated on the console:

```
Analysis of number  -9463
********************************
negative - odd - 4 digits
```

Q2-110_TestResults

Write a C-program **TestResults**, that generates a grade from a score a student has reached in a test.
Test scores are between 0 and 48. If the input score is incorrect (i.e., score < 0 or score > 48), the program displays an error message and stops. On correct entry, the grade is calculated using the following key:
45-48 Very good
39-44 Good
32-38 Average
25-31 Pass
0-24 Fail

```
Reached points: 40
A score of 40 results in grade Good.
```

Q2-111_StringAnalysis

Write a C-program **StringAnalysis**, that inputs and analyses the first 6 characters of a string. It counts the number of capital letters **numCaps**, lower case letters **numLower**, the number of digits **numDigits**, and the number of special characters **numSpecials**. It also calculates the sum of all ASCII values **asciiSum**.

```
Please enter your text: +M25+?
Analysis of the text: +M25+?
Uppercase letters: 1
Lowercase letters: 0
Digits: 2
Special charters: 3
ASCII-Sum: 329
```

Q2-112_DecimalPlaces

Write a C-program **DecimalPlaces**, that reads a positive or negative integer number with a maximum of 6 digits. It calculates how many digits the number has. If the number has more than 6 digits, it generates an error message.

```
Please enter your number: 54321
The number has 5 digits.

Please enter your number: 7654321
The number has too many digits!
```

Q2-113_Calculator

Write a C-program **Calculator**, that reads two numbers and an operator (+,-,*,/) and calculates the result.
Print the result with the invoice on the Console output.

```
Please enter the first number: 4
Please enter the arithmetic operator (+,-,*,/): +
Please enter the second number: 5
Invoice/Result: 4 + 5 = 9

Please enter the first number: 4
Please enter the arithmetic operator (+,-,*,/): /
Please enter the second number: 0
Invoice/Result: Division by zero is not allowed!
```

Q2-2 Exercises: `switch`-statements

Q2-201_ NumberGame
Write a C-program **`NumberGame`**, that inputs an integer number.
Divide the number by 5 and perform following actions.

- If the number is divisible by 5, then the number should be doubled.
- If the remainder is 2, then the number should be halved.
- If the remainder is 3, then the number should be increased by 3.
- In all other cases, the number must be reduced by 4.

```
Please enter the number: 17322
New value: 8661
```

Q2-202_ WeekDay
Write a C-program **`WeekDay`**, which determines and outputs the corresponding weekday for a date.

Note:
In 1582 the Gregorian calendar was introduced. With the help of Christian Zeller's formula, for each date from the year 1582 onwards, determine the corresponding weekday w, with Sunday being 1, Monday which corresponds to 2 etc.:

w = (t + ((m + 1) * 26 / 10) + j + j / 4 + h / 4 - 2 * h) % 7
with t = day, m = month,
h = hundreds of the year, j = year within the century.

All divisions are to be carried out in whole numbers.
Problems can arise for a few days, for example if the dividend becomes negative,
i.e. if w < 0 : add the value 7 to w.

Example: 6.7.1964 -> t = 6, m = 7, j = 64, h = 19 w = 72%7 = 2

```
Please enter a date: 6.7.1964
The 6th of July is a Monday.
```

Q2-203_ Calendar
Write a C-program **`Calendar`**, which reads a date in the form of two integers (day and month), and determines:

- what is the day of the year
- and what is the name of the month.

Assume that February has 28 days.

```
Please enter the day: 3
Please enter the month: 2

February the 3rd is day 34 of the year.
```

Q2-204_ Season

Write a C-program **Season**, which reads a date in the form of two integers (day and month) and determines the season:
Note:
21.3. . . . spring, 21.6. . . .sommer, 23.9. . . . autumn, 21.12. . . .winter.

```
Please enter the day: 3
Please enter the month: 2

On the 3rd of month 2 is winter.
```

Additionally (optional)

```
On the 3rd of February is winter.
```

Q2-205_ RomanNumbers

Write a C-program **RomanNumbers**,, that converts a three-digit Roman number to the corresponding decimal value.
Neglect the **Subtraction Rule**.
The following uppercase letters are allowed:
I, V, X, L, C, D, M for **1, 5, 10, 50, 100, 500,** and **1000**.

```
Please enter a Roman number: XXI
The Roman number: XXI is decimal 21.
```

Q2-206_ HexNumber

Write a C-program **HexNumber**,, that takes as input a two digit hexadecimal number and determines whether this number is a valid hexadecimal number.
In case the number was correct it is converted to a decimal number.
Both lowercase and uppercase letters are allowed.

```
Please enter a hexadecimal number: FF
The decimal value of hex FF is 255.

Please enter a hexadecimal number: FX
The entered number FX is not a hex number.
```

Q2-3 Exercises: Loops (Head controlled loops, while – for)

Q2-301_ NumberOutput
Write a C-program **NumberOutput**, that reads a number from the console and displays all numbers from 1 to the entered number.

```
Please enter the number: 3
1
2
3
```

Q2-302_ SquareNumbers
Write a C-program **SquareNumbers,** that reads a number from the console and displays all square numbers from 1 to the entered number.
Pay attention to the output format!

```
Please enter the number: 4
  1 -    1
  2 -    4
  3 -    9
  4 -   16
```

Q2-303_ Sum_I
Write a C-program **Sum_1,** that reads a number from the console and sums all the numbers from 1 to the entered number. (e.g., 1+2+3+4+5=15)

```
Please enter the number: 5
Sum: 15
```

Q2-304_ Sum_II
Write a C-program **Sum_2,** that reads 5 numbers from the console, adds them up and outputs the sum.

```
Enter number 1: 2
Enter number 2: 4
Enter number 3: -4
Enter number 4: -2
Enter number 5: 1
Sum: 1
```

Q2-305_ Faculty_I
Write a C-program **Faculty_1,** that reads a number from the console, computes the faculty and outputs the result (e.g., 5! = 1*2*3*4*5 = 120)

```
Please enter the number: 5
5! = 120
```

Q2-306_LotteryNumbers_I
Write a C-program **LotteryNumbers_1,** that generates numbers between 1 and 45 and displays them on the console

```
Lottery numbers 6 out of 45: 42 40 3 21 26 41
```

Q2-307_ Roll_the_dice

Create a C program `Dice_1`, that reads from the console how often the dice should be rolled. The program rolls the dice by generating a corresponding number of random numbers between 1 and 6. It should be counted, how many times each number was rolled

```
Number of throws: 1000
Number of points 1: 183
Number of points 2: 160
Number of points 3: 168
Number of points 4: 154
Number of points 5: 163
Number of points 6: 172
```

Q2-308_ TempTable

Create a C program `TempTable`, that creates a Celsius-Fahrenheit table. For temperatures in the range from -25 degrees to +40 degrees Celsius, the associated Fahrenheit values should be calculated and output as a table. Use an increment of 5 degrees Celsius for the temperature jumps.
Note: C=5/9 * (F − 32)
Output:

```
Celsius Fahrenheit table:
----------------------------------------
-25 degrees Celsius is -13 degrees Fahrenheit
-20 degrees Celsius is -4 degrees Fahrenheit
.
.
.
+35 degrees Celsius is 95 degrees Fahrenheit
+40 degrees Celsius is 104 degrees Fahrenheit
```

Q2-309_ OddNumbers

Create a C program `OddNumbers`, that outputs all odd numbers between an entered start and final value. If the end value is smaller than the start value, the two values must be swapped.

```
Starting value: 9
End value: 2
3, 5, 7, 9
```

Q2-310_ DigitSum_I

Create a C program `DigitSum_1`, that put outs all two-digit natural numbers that are greater than six times their digit sum.
Make a line break after each ten numbers.

```
10 20 21 30 31 32 40 41 42 43
50 51 52 53 60 61 62 63 64 70
71 72 73 74 75 80 81 82 83 84
85 86 90 91 92 93 94 95 96 97
```

Q2-311_ DigitSum_II

Create a C program `DigitSum_2`, that puts out all three-digit natural that are less than thirty times their digit sum. Make a line break after each ten numbers.

Q2-312_ Prime number

Create a C program `PrimeNumber`, that checks whether an entered number is a prime number or not.

```
Enter an integer: 29
The number 29 is a prime number
```

Q2-313_ Exponentiation

Write a program `Exponentiation`, that takes the base x and the exponent y of a power and calculates the power x^y.
The program should return correct results when the exponent is positive, negative, or equal to zero.
The pow() function must NOT be used!

```
2 ^ 3 = 8
2 ^ (-3) = 1/8 = 0.125
2 ^ 0 = 1
```

Q2-314_ Freefall

Write a program `Freefall`, which creates a table listing fall height and fall time. the
The upper and lower limit of the drop height as well as the increment are entered by the user.

The formula for the drop height is: $h = \frac{1}{2} g t^2$

$g = 9.81 m/s^2$ (gravitational acceleration)

```
Freefall
-----------
height upper limit: 100
Height lower limit:0
Increment (in meters):25
```

Output:

```
Height of fall [m]     | fall time [s]
----------------------|-------------
                    0  | 0.00
                   25  | 2.26
                   50  | 3.19
                   75  | 3.91
                  100| 4.52
```

Q2-315_ Armstrong numbers

Write a program `ArmstrongNumbers`, which calculates and outputs all three-digit Armstrong numbers.
The following calculation formula applies to three-digit Armstrong numbers:

$$HZE = H^3 + Z^3 + E^3$$

e.g.: $153 = 1^3 + 5^3 + 3^3$

Q2-316_ PI_Calculation

The following gentlemen have each developed different approximation methods for calculating the circle number π:

John Wallis (1656):

$$\frac{\pi}{2} = \frac{2}{1} \cdot \frac{2}{3} \cdot \frac{4}{3} \cdot \frac{4}{5} \cdot \frac{6}{5} \cdot \frac{6}{7} \cdot \frac{8}{7} \cdot \ldots$$

Gottfried Wilhelm Leibniz (1682):

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots$$

Leonhard Euler (1748):

$$\frac{\pi^2}{6} = \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} + \cdots$$

Write a program `PiCalculation`, that implements these formulas as an algorithm, reading in the number of elements. Try to rate these formulas for accuracy.
The output can look like this:

```
Comparison of approximation methods for calculating the circle
number Pi
------------------------------------------------- --------------
Added elements: 1000

procedure | Deviation from M_PI
----------|--------------------
Wallis    | 0.001569
Leibniz   | 0.001000
Euler     | 0.000955
```

Q2-317_ TowerCalculation

Write a program `Tower`, that reads in a starting number for a tower calculation and then performs the following calculation.

```
starting number: 3

      3 * 1 = 3
      3 * 2 = 6
      . .
      . .
  15120 * 8 = 120960
 120960 * 9 = 1088640
1088640 / 1 = 1088640
1088640 / 2 = 544320
 544320 / 3 = 181440
    . .
    . .
      27 / 9 = 3
```

Q2-318_ FindDivider
Write a program `FindDivider,` which calculates and outputs all divisors of a number.

```
Enter an integer: 75
The number 75 is divisible by the following numbers:
1 3 5 15 25 75

Enter an integer: 17
The number 17 is divisible by the following numbers: 1 17
```

Q2-319_ DeleteRelatedNumbers
Write a program `NumberRelationship,` which has the following functionality:
For an entered digit, the numbers from 0 to 99 are output from a 10x10 table, with the digit related numbers are replaced with dashes.
A numerical relationship results from:
• The tens place of the number is equal to the digit
• One place of the number is equal to the digit
• The checksum of the number is equal to the digit
• The number is a multiple of the digit

```
Enter a digit between 1 and 9: 7
||  1  2  3  4  5  6 ||  8  9
10 11 12 13 || 15 || || 18 19
20 || 22 23 24 || 26 || || 29
30 31 32 33 || || 36 || 38 39
40 41 || || 44 45 46 || 48 ||
50 51 || 53 54 55 || || 58 59
60 || 62 || 64 65 66 || 68 69
|| || || || || || || || || ||
80 81 82 83 || 85 86 || 88 89
90 || 92 93 94 95 96 || || 99
```

Q2-320_ NumberTester
Write a program `NumberTester,` that finds all three-digit numbers that are divisible by all of their digits.

For the better understanding:
number = 126 -> divisible by 1, divisible by 2, divisible by 6

The program should produce the following output:

```
Number found: 111
Number found: 112
...
```

Q2-321_ DigitSum_III
Write a program `DigitSum_3,` which reads an arbitrarily large integer and calculates the sum of its digits.

```
Please enter a whole number: 4294967295
Digit sum: 57
```

Q2-322_ Count characters
Write a program `CountCharacters,` which reads any number of characters via the console and at the end the number of characters entered.
The input is terminated with Ctrl-Z.

Q2-323_Input statistics

Write a program `InputStatistics,` that reads any text character by character and counts the number of all characters, every word and every line.
The input is terminated with Ctrl-Z. As a separation between words are all spaces in any number. Spaces are not counted in the statistics.

```
Was once a boomerang;
Was a little too long.
Boomerang flew a piece,
but didn't come back.
Audience - for hours -
Waiting for boomerang.
^Z

Number of characters entered: 112
Number of words entered: 25
Number of rows: 6
```

Q2-324_ Analyze characters

Write a program `AnalyzeCharacters,` that counts and outputs the number of uppercase letters, lowercase letters, consonants, vowels, digits, punctuation marks and spaces of a text entered character by character.
Completed the input is replaced by the EOF character (Ctrl-Z)

```
Input:
The cinema ticket costs 6 euros, which is cheap.

Capital letters: 1
Lower case: 36
Consonants: 23
vowels:14
Digits: 1
Punctuation marks: 2
Space: 8
```

Q2-325_ Decimal to binary converter

Write a program `DecBinCon,` which reads an integer and converts it from decimal to binary and outputs it.

```
Number in decimal system:    35
Number in binary system: 100011
```

Q2-326_ Prime factorization

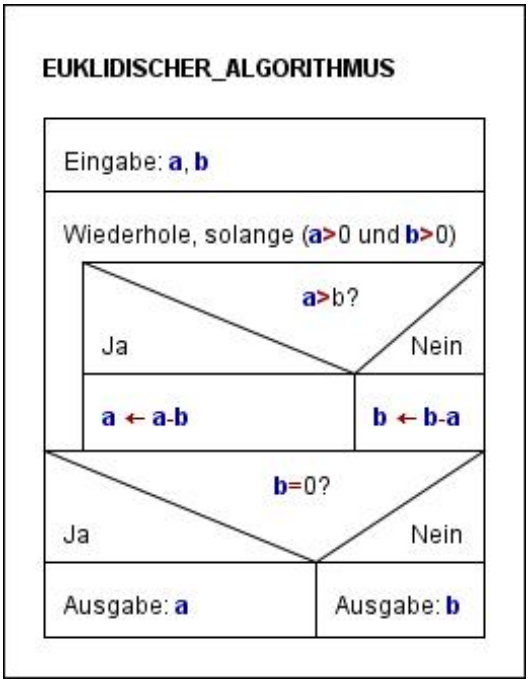
Write a program `PrimeFactorization,` which reads a number and breaks it down into its prime factors.

```
Any integer: 36
Prime factors: 2 2 3 3

Any integer: 17
Prime factors: 17
```

Q2-327_ Greatest common divisor

Write a program `GreatestCommonDivisor`, that reads two numbers and calculates the greatest common divisor. Implement the Euclidean algorithm shown in the following structogram.



Q2-4 Exercises: Loops (Foot controlled loop, do – while)

Q2-401_ Dice game_I

Write a program `DicGame_1,` that keeps throwing the dice as long as the **total** is < 100. There is to count how many times the dice were rolled. The rolling of the dice is again done by random number generation in the interval [1;6] implemented.

```
With 29 throws the total of 101 was reached.
```

Q2-402_ Dice game_II

Write a program `DicGame_2,` which counts how many times the dice must be rolled until 10 sixes have been rolled.

```
For 10 sixes, the dice had to be rolled 67 times.
```

Q2-403_ Fibonacci series

Write a program `FibonacciSeries,` that calculates the Fibonacci series up to a read number n. A number always results from the sum of the two predecessors.

1, 1, 2, 3, 5, 8, 13, 21, 34, ..., n
Note: The Fibonacci series starts with 1 and 1.

Q2-404_ Plausibility check

Write a program `PlausibilityCheck,` which repeats entering an integer value z as long as z is outside the interval [−10, +10].

```
Enter a value in the interval [-10,+10]: 50
Enter a value in the interval [-10,+10]: -50
Enter a value in the interval [-10,+10]: -10
```

Q2-405_ Sum calculation

Write a program `SumCalculation,` which reads in an integer positive sum value. The program should determine the number of integer positive summands starting with 1 that are added until the total has exceeded the entered total value.
E.g.: total value 25 -> 1 + 2 + 3 + 4 + 5 + 6 + 7 = 28

```
Total value: 25
Number of summands: 7
Calculated total: 28
```

Addition: Output the summands and the calculated sum in the following form:

```
1 + 2 + 3 + 4 + 5 + 6 + 7 = 28
```

Q2-406_ Random capital letter

Write a program `RandomCapitalLetter,` which generates random capital letters and displays them one after the other on the screen. If an 'X', 'Y' or 'Z' is generated, the program is terminated.

```
MFNKUZ
```

Q2-407_ Random numbers

Write a program `RandomNumbers`, which generates random numbers in the interval [−50 + 50]. The negative and positive random numbers are to be summed separately until the amount of one of the two sums is >= 15000.
The program also counts the number of generated random numbers.

```
random numbers
-------------
Number of positive random numbers: 543
Sum of positive random numbers: 14131
Number of negative random numbers: 596
Sum of negative random numbers: -15011
```

Q2-408_ Capital calculator

Write a program `CapitalCalculator`, that calculates how many years a share capital has to earn interest, so that the resulting total capital is at least twice the share capital.
The following formula can be used for the calculation:
n ... number of years

$$\text{Capital} = \text{share capital} \cdot \left(1 + \frac{\text{interest rate}}{100}\right)^n$$

```
capital calculator
--------------
Share capital: 1000.00
Interest rate: 5.00
After 16 years the capital is 2078.93
```

Q2-409_ Rectangular area

Write a program `RectangularArea`, which reads sides a and b of a rectangle. Alternately halve the sides until the resulting area is < 1/20 of the original area.

```
rectangular area
---------------
Side length a = 36.8
Side length b = 19.7
Rectangular area at the beginning: 724.96
Rectangle area after bisections: 22.66:
Side length a: 4.60 after 3 bisections.
Side length b: 4.93 after 2 bisections.
```

Q2-410_ Heron's square root

Write a program `HeronSquareRoot`, which approximates the square root of a using the following Heron method:

$$x_{i+1} = \frac{1}{2}\left(x_i + \frac{a}{x_i}\right)$$

The first approximation x_1 is a itself.

The sequence of approximate values shall be calculated until two consecutive values differ by less differ than 0.000001.

Q2-411_ Heron's k-th root

Write a program `HeronKRoot,` which approximates the k-th root of a using the following Heron method:

$$x_{i+1} = \frac{(k-1)x_i^k + a}{kx_i^{k-1}}$$

The first approximation x_1 is a itself.
The sequence of approximate values shall be calculated until two consecutive values differ by less differ than 0.000001.

Q2-412_ Guess numbers_I

Write a program `GuessNumbers_1,` which generates a random number between 1 and 10. The user has max 3 attempts to guess the number. If the number is wrong, the program should help the user indicate whether the number to be guessed is larger or smaller.
Output:
If the number was guessed: number of attempts
If the number was not guessed: the number to guess

```
Guess a number in the range 1 to 10 (3 tries):
6
The number is bigger.
8
The number is smaller.
7
congratulations You guessed the number in 3 try(s).
```

Q2-413_ Guess numbers_II

Write a program `GuessNumbers_2,` designed to guess a number that the user has made up. The user thinks of a number between 1 and 100 that the computer should guess. The user gives the computer a helping hand by typing 'h' for higher after each number determined by the computer, 'l' for lower and enter 'g' for guess. Finally, the guessed number and the number of attempts should be given

Solution variant 1: The computer generates a random number within the range limited by user feedback.
Solution variant 2: The computer suggests the median value of the delimited by user feedback range (divide and conquer).

Questions:
1. Is there a variant of the solution that is more fun to play than the other? If yes why?
2. How do the two solution variants differ in terms of the number of attempts? What is each the maximum value in the attempts?

Q2-5 Exercises: Nested Loops

Q2-501_ Faculty_II

Write a C-program `Faculty_2,` which reads two positive integers a, b in the range 1 to 12. Calculate the factorial of all numbers that are in the interval [a..b] and display them on the console.

```
Calculating the factorials of an interval [a,b]
-----------------------------------------------
a: 5
b: 8
5! = 120
6! = 720
7! = 5040
8! = 4032
```

Q2-502_ Pythagorean triple

Write a C-program `PythagoreanTriple,` that finds all Pythagorean triples up to a specified upper limit.

A Pythagorean triple is formed by three integers, the lengths of the sides of a right triangle are. This means that the Pythagorean theorem applies to them. Make sure that the elements of the triple (x, y, z) sorted in ascending order.

```
Enter an upper limit for searching Pythagorean triples: 25
The Pythagorean triples are:
(3, 4, 5)
(6, 8, 10)
(5, 12, 13)
(8, 15, 17)
(12, 16, 20)
(7, 24, 25)
```

Q2-503_LotteryNumbers_II

Write a C-program `LotteryNumbers_2,` that generates six different numbers between 1 and 45 and displays them on the console

Additional task: Give out the lottery numbers sorted in ascending order.

```
  Lottery numbers 6 out of 45: 3 9 13 21 29 42
```

.